Lab 4: Word Embeddings

Part A: Exploring Static Word Embeddings

Part A of this lab lets you practice working with static neural word embeddings. We'll use Python libraries that learn static word embeddings, perform vector operations on word vectors, and create and manipulate contextual embeddings. Most data for this lab are obtained from the file lab04.zip on Canvas.

You may run your code on any platform you like. Colab (`colab.research.google.com`) is a reasonable choice. This notebook should run on Colab without modification. To avoid running into resource limits in future labs, you might consider getting a paid Colab account for the duration of the course. You shouldn't need a paid account for this lab, but once we are making heavier use of the GPU, a paid account will reduce your headaches. And it's far cheaper than a textbook.

We'll use the following libraries. Many are pre-loaded on Colab, but if you're doing this on your own machine you might need to pip install them.

Import the gensim package first, to avoid versioning errors in Colab.

```
import gensim
```

Now import the remaining packages.

```
import gdown
import gzip
import nltk
import numpy as np
import os
import spacy
import umap

from gensim.models import Word2Vec, KeyedVectors
from google.colab import drive
from matplotlib import pyplot as plt
```

When using Colab, we will load our data files from Google drive. When mounting our drive the first time, Google will verify ownership of the drive before granting access to it.

```
gdrive_mount = '/content/drive'
drive.mount(gdrive_mount, force_remount=True)
```

```
Mounted at /content/drive
```

Create directories for our data and models. Note that we need to put double quotes around the directory name to run this in a Notebook because in their infinite wisdom Google has placed a space in the default gdrive path.

```
# Change this path if you'd like to work on your lab in a different directory:
labdir = '/content/drive/My Drive/NLPCourse/Lab4'
!mkdir -p "$labdir"
!mkdir -p "$labdir/data"
!mkdir -p "$labdir/models"
%cd "$labdir"
!ls
```

```
/content/drive/My Drive/NLPCourse/Lab4
data  models
```

```
output_filepath = 'data/GoogleNews-vectors-negative300.bin'
compressed_output_path = 'data/GoogleNews-vectors-negative300.bin.gz'
file_id = '0B7XkCwpI5KDYNlNUTTlSS21pQmM'

# Check if the uncompressed file already exists
if not os.path.exists(output_filepath):
    print("GoogleNews vectors not found, downloading and extracting...")

    # Download the compressed file
    gdown.download(id=file_id, output=compressed_output_path, quiet=True)

    # Ungzip the file
    try:
        with gzip.open(compressed_output_path, 'rb') as f_in:
            with open(output_filepath, 'wb') as f_out:
                f_out.writelines(f_in)
        print("Download and extraction complete.")
    except Exception as e:
        print(f"Error during extraction: {e}")
    finally:
        # Remove the compressed file to save space, even if extraction failed
        if os.path.exists(compressed_output_path):
            os.remove(compressed_output_path)
else:
    print("GoogleNews vectors already exist.")
```

```
GoogleNews vectors not found, downloading and extracting...
Download and extraction complete.
```

You should download the file small.txt.zip from Canvas, unzip it, and place the resulting file in the data directory. You should also obtain a set of pre-calculated embedding vectors called `GoogleNews-vectors-negative300.bin.gz` from here, ungzip them, and place them in the data directory. If you are using Colab, you can do this directly without downloading it to your machine as follows:

We'll use SpaCy to tokenize our text. There are 50,000 lines in small.txt, so this may take a while. To speed things up a bit, we'll turn off some of SpaCy's features.

```
nlp = spacy.load('en_core_web_sm', disable = ['ner', 'parser'])
```

We'll treat every line as a sentence to be tokenized. We'll run SpaCy's tokenizer over the line, then pull out the text of the tokens (ignoring punctuation). We will end up with a list of lists, one per line, where each contained list is a list of the words in the sentence as strings.This will take four or five minutes.

```
with open('data/small.txt', 'r') as infile:
  collection = [[token.text for token in nlp(line.lower()) if not token.is_punct] for line in infile]
```

Safety check to make sure our data was loaded correctly:

```
assert len(collection) == 50000, "Load of small.txt failed"
print(collection[42])
```

```
['you', "'ve", 'literally', 'taken', 'on', 'the', 'energy', 'of', 'this', 'dear', 'one', 'in', 'your',
```

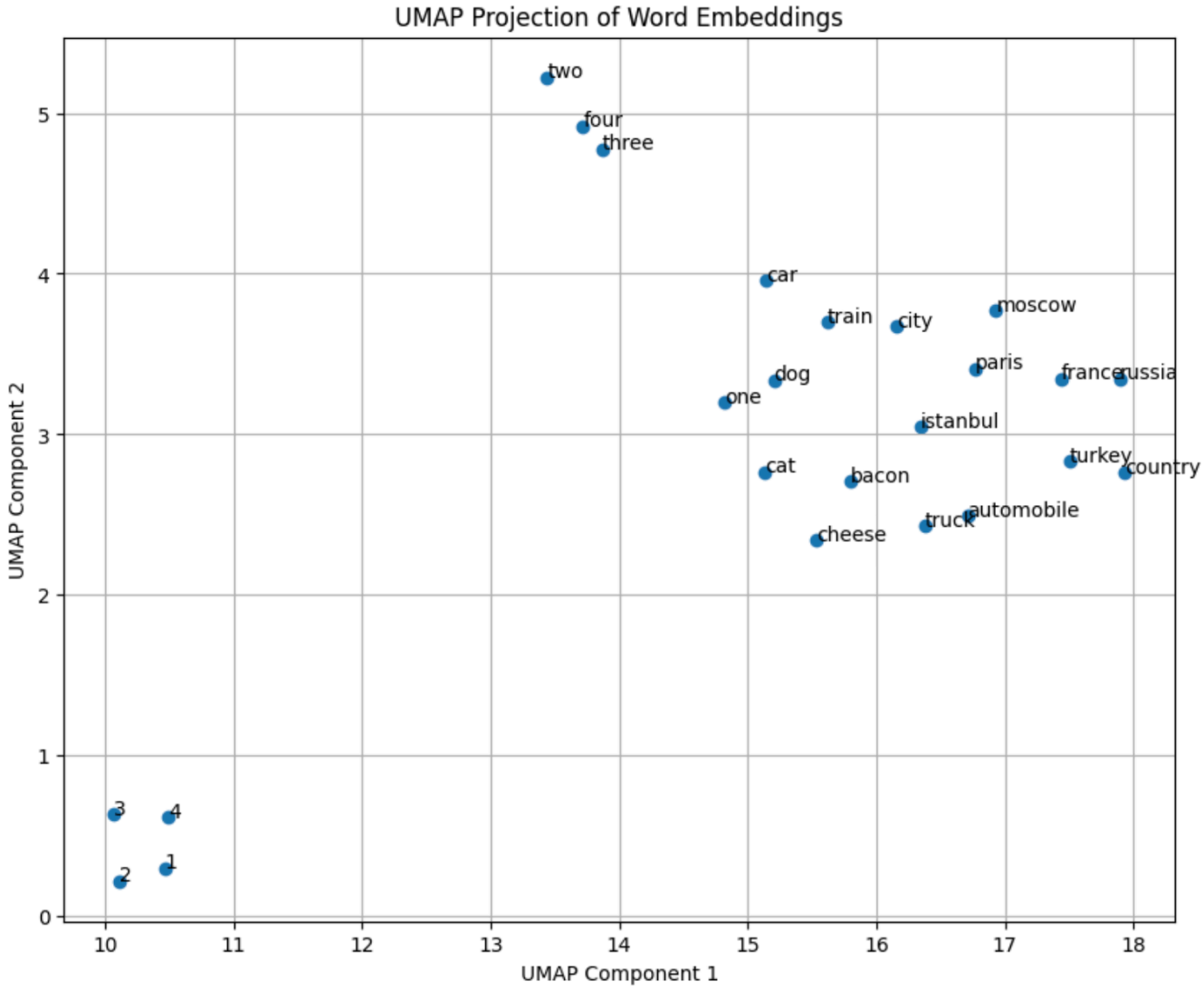
Build a 100-dimensional Word2Vec model, using gensim's implementation of Word2Vec.

```
model = Word2Vec(sentences=collection, vector_size=100, window=5, min_count=2, sg=1, negative=8)
print(model)
```

```
Word2Vec<vocab=26733, vector_size=100, alpha=0.025>
```

```
words_to_visualize = ['paris', 'istanbul', 'moscow', 'france', 'turkey', 'russia',
                      'cat', 'dog', 'chicken', 'duck',
                      'truck', 'train', 'car', 'cars', 'automobile', 'automobiles',
                      'one', 'two', 'three', 'four', '1', '2', '3', '4',
                      'cheese', 'bacon', 'tofu', 'hummus']
```

(10 points) To visualize the vector relationships between related words, make a 2-dimensional projection using UMAP (Uniform Manifold Approximation and Projection). UMAP is a dimension reduction technique like PCA, but it is a non-linear method that captures both local and global data structures. Create a scatter plot for words_to_visualize. You may use the umap-learn package, imported above, or your own favorite package.



Your plot may look different. However, semantically related words should be near each other in this two-dimensional representation. You may also notice that the vectors for (france - paris) and (russia - moscow) are similar.

```
wordVectors=[]
words=[]
```

```
for word in words_to_visualize:
    if word in model.wv:
```

```
            wordVectors.append(model.wv[word])
            words.append(word)
```

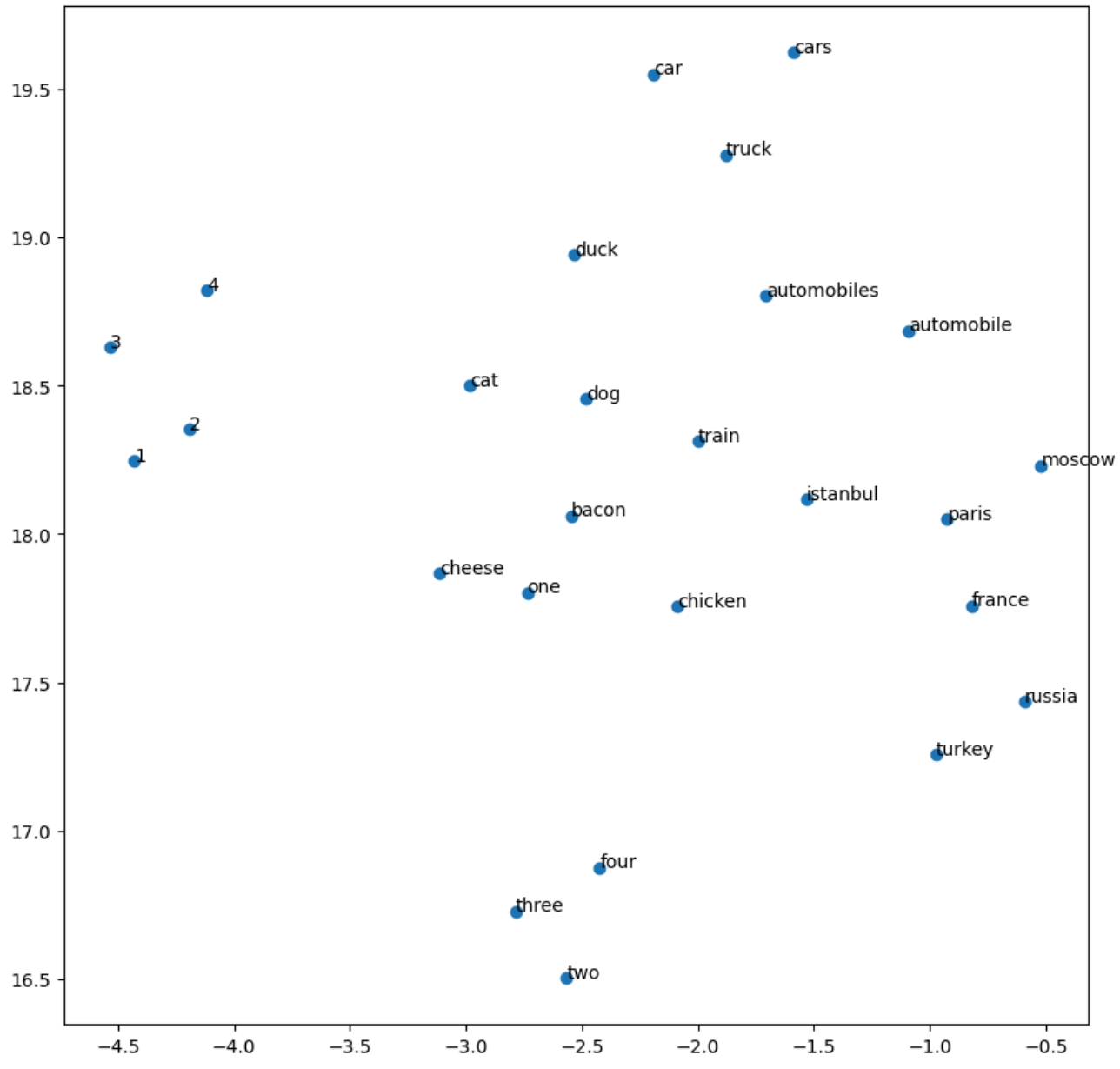
```
wordVectors = np.array(wordVectors)
```

```
vectorReducer=umap.UMAP(n_components=2, random_state=42)
```

```
reducedEmbedding = vectorReducer.fit_transform(wordVectors)
```

```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1
  warn(
```

```
plt.figure(figsize=(10, 10))
plt.scatter(reducedEmbedding[:,0], reducedEmbedding[:,1])
for i, word in enumerate(words):
    plt.annotate(word,xy=(reducedEmbedding[i, 0], reducedEmbedding[i, 1]))
```



Better quality embeddings can be obtained from larger text. Let's load vectors built off of a much larger collection: the Google News pretrained embeddings. We will wipe out our original model to preserve memory. Note that this model is not

case-normalized.

```
model = KeyedVectors.load_word2vec_format("data/GoogleNews-vectors-negative300.bin", binary=True)
```

We can verify that the model contains word vectors.

```
print(model['purple'])
```

```
[ 1.21093750e-01 -4.68750000e-02  3.58886719e-02  2.83203125e-01
 -1.66015625e-01 -8.05664062e-02  6.17675781e-02 -4.84375000e-01
 -9.47265625e-02 -4.88281250e-03 -1.34765625e-01  1.13281250e-01
  2.12890625e-01  9.91210938e-02 -1.34765625e-01 -2.25585938e-01
 -2.42187500e-01  1.56250000e-01 -6.98242188e-02 -2.18750000e-01
 -1.47460938e-01  2.98828125e-01 -1.01928711e-02 -2.21679688e-01
 -2.25585938e-01 -3.17382812e-02 -1.73828125e-01  4.68750000e-02
  2.50244141e-02 -2.21679688e-01 -4.39453125e-02  2.41210938e-01
  1.25976562e-01  1.62353516e-02 -1.08398438e-01  6.83593750e-02
  3.63281250e-01 -4.94384766e-03 -8.83789062e-02  2.22656250e-01
  7.32421875e-02 -8.34960938e-02 -4.71191406e-02 -2.69531250e-01
  1.26953125e-01 -8.30078125e-02  9.33837891e-03 -2.46093750e-01
 -7.56835938e-02  1.45507812e-01 -1.92382812e-01  1.26953125e-01
  1.85546875e-01 -3.05175781e-02  4.78515625e-02  4.73022461e-03
 -1.25000000e-01 -1.16210938e-01 -7.37304688e-02 -8.30078125e-02
 -1.94335938e-01  1.11816406e-01 -8.44726562e-02  9.81445312e-02
 -1.35742188e-01 -6.17675781e-02 -1.06933594e-01  8.00781250e-02
 -3.73535156e-02  3.28125000e-01  2.50000000e-01  1.92382812e-01
 -1.40625000e-01 -1.75781250e-01 -1.01562500e-01 -6.10351562e-02
  2.40234375e-01 -1.69677734e-02 -3.90625000e-02  5.58593750e-01
  2.67578125e-01 -2.49023438e-01 -1.24511719e-01 -3.35937500e-01
 -2.45117188e-01 -1.14746094e-01 -1.95312500e-01  1.09375000e-01
 -1.42822266e-02  2.58789062e-02 -1.35742188e-01  3.93066406e-02
 -1.42822266e-02  1.37695312e-01  5.68847656e-02  4.98046875e-02
  1.45507812e-01  2.20947266e-02  8.66699219e-03  6.83593750e-02
 -1.06933594e-01 -1.67968750e-01  2.49023438e-01 -2.50000000e-01
 -2.73437500e-01 -5.15136719e-02  4.02343750e-01 -9.61914062e-02
 -6.39648438e-02 -1.35742188e-01 -9.81445312e-02 -1.30859375e-01
  4.05273438e-02  2.42187500e-01  3.45703125e-01  3.75976562e-02
 -6.00585938e-02  5.68847656e-02  9.86328125e-02 -2.09045410e-03
 -1.48925781e-02 -5.56640625e-02  4.80957031e-02 -2.79296875e-01
  1.23046875e-01  9.52148438e-02  2.50244141e-02  8.00781250e-02
  1.49414062e-01  5.61523438e-02 -2.50000000e-01  1.22070312e-01
 -1.56250000e-02  1.03027344e-01  9.08203125e-02  3.35937500e-01
 -1.88476562e-01 -6.31713867e-03  2.33459473e-03  4.00390625e-02
 -8.83789062e-02  9.91210938e-02 -9.08203125e-02 -2.48046875e-01
  1.92382812e-01  2.27539062e-01  5.61523438e-03 -8.10546875e-02
 -2.30468750e-01 -2.01171875e-01  1.80664062e-01  1.20605469e-01
 -1.23535156e-01  9.03320312e-02  5.21850586e-03 -1.32812500e-01
 -3.95507812e-02 -2.00195312e-01 -1.49414062e-01  1.38671875e-01
  1.11694336e-02 -5.83496094e-02  4.84375000e-01  9.81445312e-02
 -1.25976562e-01 -1.08032227e-02  7.03125000e-02 -1.81640625e-01
 -2.17773438e-01  5.34667969e-02 -1.07421875e-01 -3.63769531e-02
 -2.14843750e-01  3.33984375e-01 -2.26562500e-01 -9.32617188e-02
  2.86865234e-02 -1.80664062e-01  2.44140625e-02 -2.32421875e-01
 -1.72851562e-01  4.85839844e-02 -2.71484375e-01  7.08007812e-02
 -1.33789062e-01 -2.79296875e-01  1.18164062e-01  1.82617188e-01
 -5.90820312e-02  5.55419922e-03  5.76171875e-02 -1.35742188e-01
 -1.73828125e-01 -8.78906250e-02  1.48437500e-01  9.13085938e-02
  9.15527344e-05  6.73828125e-02  9.58251953e-03 -5.46875000e-02
  4.15039062e-02 -1.08398438e-01 -2.63977051e-03 -8.69140625e-02
  8.44726562e-02  9.64355469e-03  8.83789062e-02  7.78198242e-03
 -6.78710938e-02 -1.66992188e-01 -5.44433594e-02 -2.41699219e-02
 -1.67968750e-01 -9.81445312e-02  2.30468750e-01  1.06445312e-01
  1.56250000e-01 -1.68945312e-01 -1.19628906e-01  1.95312500e-01
 -1.32812500e-01 -2.44140625e-02 -1.52343750e-01  1.26953125e-01
 -7.42187500e-02 -6.83593750e-02  1.86767578e-02  6.34765625e-02
  4.17480469e-02 -2.22656250e-01  1.19628906e-01  2.95410156e-02
```

Explore the gensim API. Note that sometimes Python complains about the presence of the `.wv` in the following invocations; other times it complains about its absence.

```
model[x]: vector for word x
model.wv.similarity(x, y): cosine similarity between vectors for x and y
model.wv.distance(x, y): 1 - cosine similarity of x and y
model.wv.most_similar(positive=[x, y, z], negative=[a, b, c], topn=k):  return k-most similar
model.wv.most_similar_cosmul: like most_similar with a different function for three word analogies
```

To find the words most similar to wine:

```
model.wv.most_similar (positive=['wine'])
```

(2 points) Use the API to find the words most similar to 'fascinating,' 'cultivate,' and 'eggplant,' and report your results. Is any of the most similar words a reasonable synonym or antonym for the input word?

(2 points) Do you believe that the similarity / distance functions are intuitive? For example, are pairs like mother/father closer than mother/ocean? Explain your reasoning.

(2 points) According to the API, which words best complete the analogy, puppies is to dog as X is to cat?

(2 points) A classic example is queen = (king - man) + woman. Find two other interesting analogies using the API.

(5 points) A pair of words might be synonyms, antonyms, similar, related, subordinate/superordinate, or unrelated. Identify at least one pair of words in each category, and compare their embeddings. For example, you might compare plots, or compare cosines or L2 distances, etc. Given a pair of words, do you think it is possible to determine whether they are a) synonymous, b) antonymous, c) similar, d) related, e) subordinate/ superordinate, or f) unrelated, using only their embedding vectors? Give your reasoning for each category of relationship.

(2 points) Record any interesting observations, and report any examples you like of good or dubious performance.

```
similar = model.most_similar(positive=['fascinating'])
```

```
similar
```

```
[('interesting', 0.7623067498207092),
 ('intriguing', 0.7245113253593445),
 ('enlightening', 0.6644250154495239),
 ('captivating', 0.6459898352622986),
 ('facinating', 0.6416683793067932),
 ('riveting', 0.6324825286865234),
 ('instructive', 0.6210989356040955),
 ('endlessly_fascinating', 0.6188612580299377),
 ('revelatory', 0.6170244216918945),
 ('engrossing', 0.6126049160957336)]
```

Fascinating and interesting, intriguing, captivating are good synonyms

```
similar=model.most_similar(positive=['cultivate'])
```

```
similar
```

```
[('cultivating', 0.7274795174598694),
 ('cultivated', 0.6766660809516907),
 ('nurture', 0.6695472002029419),
 ('cultivates', 0.6159787178039551),
 ('develop', 0.6053071618080139),
 ('Cultivate', 0.5910324454307556),
 ('Cultivating', 0.566783607006073),
 ('nuture', 0.5440779328346252),
 ('nourish', 0.5328434109687805),
 ('grow', 0.5282535552978516)]
```

cultivate similar words give a lot of different variations of the words for example cultivating and cultivated. Nurture and develop are good synonyms for cultivate.

```
similar=model.most_similar(positive=['eggplant'])
```

```
similar
```

```
[('zucchini', 0.7264777421951294),
 ('cauliflower', 0.6968439221382141),
 ('eggplants', 0.6966414451599121),
 ('bok_choy', 0.6907373070716858),
 ('bell_peppers', 0.6875451803207397),
 ('broccoli_rabe', 0.6864162087440491),
 ('bell_pepper', 0.685966432094574),
 ('asparagus', 0.6851089596748352),
 ('pesto', 0.6766839623451233),
 ('escarole', 0.6759323477745056)]
```

zuchini and cauliflower are not synonyms of egglplant but they are related.

```
pairs=[
    ('mother','father'),
    ('mother','ocean'),
    ('queen','king'),
    ('queen','apple')
]
```

```
for word1, word2 in pairs:

  sim = model.similarity(word1,word2)
  dist = model.distance(word1,word2)
  print(f"{word1},{word2}   {sim}   {dist}")
```

```
mother,father   0.7901482582092285   0.20985174179077148
mother,ocean   0.10562344640493393   0.8943765535950661
queen,king   0.6510956883430481   0.3489043116569519
queen,apple   0.11686956882476807   0.8831304311752319
```

The similarity and distance functions are intuitive. As seen from above, mother and father are similar and don't have much distance between them, however mother and ocean are not so similar and have a lot of distance between them. The same goes for queen and king, they are close and not as distance, whereas queen and apple aren't so similar and are far away.

```
analagyResult=model.most_similar(positive=['cat','puppies'], negative=['dog'])
```

```
analagyResult
```

```
[('kittens', 0.7716175317764282),
 ('cats', 0.735849142074585),
 ('kitten', 0.6659281253814697),
 ('pups', 0.6466317176818848),
 ('felines', 0.6201465129852295),
 ('kitties', 0.6081463694572449),
 ('puppy', 0.6058371067047119),
 ('Chihuahuas', 0.600016713142395),
 ('newborn_kittens', 0.588977575302124),
 ('pup', 0.5851237773895264)]
```

For the analogy, "puppies is to dog as X is to cat?",the api tells me that kittens is X which makes sense

```
result=model.most_similar(positive=['Moscow','France'],negative=['Paris'])
```

```
result
```

```
[('Russia', 0.834155797958374),
 ('Ukraine', 0.7043537497520447),
 ('Belarus', 0.6752798557281494),
 ('Russian', 0.6514027714729309),
 ('Kremlin', 0.6162627935409546),
 ('Moldova', 0.6150105595588684),
 ('Poland', 0.6035367846488953),
 ('Estonia', 0.6016945838928223),
 ('Kazakhstan', 0.5983687043190002),
 ('ex_Soviet', 0.5964206457138062)]
```

```
result=model.most_similar(positive=['broccoli','fruits'],negative=['apples'])
```

```
result
```

```
[('veggies', 0.5554287433624268),
 ('vegetables', 0.5502044558525085),
 ('cruciferous_vegetables', 0.5479790568351746),
 ('dark_leafy_greens', 0.5443115830421448),
 ('leafy_veggies', 0.5247859954833984),
 ('cruciferous', 0.5222920179367065),
 ('Cruciferous_vegetables', 0.5168887376785278),
 ('fruits_vegetables', 0.5116195678710938),
 ('leafy_vegetables', 0.5104649662971497),
 ('Broccoli_sprouts', 0.5097735524177551)]
```

A classic example is queen = (king - man) + woman. Find two other interesting analogies using the API. Two other interesting are shown above Russia = (France-Paris) + moscow veggies = (fruits-apples) + broccoli

Double-click (or enter) to edit

```
pairs=[('quick','fast'),('large','big'),('happy','cheerful')]
```

```
print("Synonyms")
for w1,w2 in pairs:
  sim = model.similarity(w1,w2)
  dist = model.distance(w1,w2)
  print(f"{w1},{w2}   {sim}   {dist}")
```

```
Synonyms
quick,fast   0.5701605677604675   0.42983943223953247
large,big   0.5561478734016418   0.44385212659835815
happy,cheerful   0.3837738335132599   0.6162261664867401
```

```
pairs=[('up','down'),('day','night'),('in','out')]
```

```
print("Antonyms")
for w1,w2 in pairs:
  sim = model.similarity(w1,w2)
  dist = model.distance(w1,w2)
  print(f"{w1},{w2}   {sim}   {dist}")
```

```
Antonyms
up,down   0.6396992206573486   0.36030077934265137
day,night   0.507000744342804   0.49299925565719604
in,out   0.35711318254470825   0.6428868174552917
```

```
pairs=[('apple','orange'),('goat','cow'),('car','motorcycle')]
```

```
print("Similar")
for w1,w2 in pairs:
  sim = model.similarity(w1,w2)
  dist = model.distance(w1,w2)
  print(f"{w1},{w2}   {sim}   {dist}")
```

```
Similar
apple,orange   0.3920346200466156   0.6079653799533844
goat,cow   0.6359611749649048   0.3640388250350952
car,motorcycle   0.6256055235862732   0.3743944764137268
```

```
pairs=[('car','gasoline'),('typing','laptop'),('sleep','blanket')]
```

```
print("Related")
for w1,w2 in pairs:
  sim = model.similarity(w1,w2)
  dist = model.distance(w1,w2)
  print(f"{w1},{w2}   {sim}   {dist}")
```

```
Related
car,gasoline   0.30619531869888306   0.6938046813011169
typing,laptop   0.3513911962509155   0.6486088037490845
sleep,blanket   0.22994039952754974   0.7700596004724503
```

```
pairs=[('car','sedan'),('fruit','mango'),('beverage','milk')]
```

```
print("Subordinate/Superordinate")
for w1,w2 in pairs:
  sim = model.similarity(w1,w2)
  dist = model.distance(w1,w2)
  print(f"{w1},{w2}   {sim}   {dist}")
```

```
Subordinate/Superordinate
car,sedan   0.6336701512336731   0.3663298487663269
fruit,mango   0.6631807088851929   0.33681929111480713
beverage,milk   0.343085914850235   0.656914085149765
```

```
pairs=[('grape','shirt'),('beef','blue'),('exit','needle')]
```

```
print("Unrelated")
for w1,w2 in pairs:
  sim = model.similarity(w1,w2)
  dist = model.distance(w1,w2)
  print(f"{w1},{w2}   {sim}   {dist}")
```

```
Unrelated
grape,shirt   0.09941279143095016   0.9005872085690498
beef,blue   -0.007134534418582916   1.007134534418583
exit,needle   0.02656792663037777   0.9734320733696222
```

Given these different types of pair words, you can sometimes tell which type they are. Synonyms and antonyms, similar, and subordinate/ superordinate all have similar similarities and distances where you won't really be able to tell between them. For related, the pairs usually have 0.3 similarity and 0.6 distance. Unrelated have very low similarity and high distance.

```
result=model.most_similar(positive=['brother','sister'],negative=['nephew'])
```

```
result
```

```
[('sisters', 0.6672795414924622),
 ('twin_sister', 0.64035964012146),
 ('mother', 0.6048528552055359),
 ('siblings', 0.602859616279602),
 ('sibling', 0.5984649062156677),
```

```
 ('daughter', 0.5945715308189392),
 ('cousin', 0.5772751569747925),
 ('aunt', 0.5634581446647644),
 ('eldest_daughter', 0.5535686612129211),
 ('younger_brother', 0.546487033367157)]
```

Dubious performance: Brother is to sister as niece is to nephew. However the output didn't have niece in it

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
result=model.most_similar(positive=['pot'])
```

```
result
```

```
[('Pot', 0.6931390762329102),
 ('pots', 0.6391127109527588),
 ('marijuana', 0.6314845085144043),
 ('cannabis', 0.5680527091026306),
 ('marijauna', 0.5500227212905884),
 ('marijuna', 0.5434213876724243),
 ('Marijuana', 0.536467969417572),
 ('medicinal_marijuana', 0.5316340327262878),
 ('marajuana', 0.5295785069465637),
 ('director_Vincent_Palazzotto', 0.522117555141449)]
```

Good Performance: It's impressive when I put pot and it gave marijuana and cannibas which means it uses different meanings of, even lesser known meanings, of words.

Part B: Exploring Contextual Word Embeddings

Now we will compare the static embeddings of Part A with contextual embeddings produced by SpaCy. The imports from Part A cover everything we need here.

SpaCy automatically builds a contextual embedding for each token it processes. Here is a function that returns a list of embedding vectors and the corresponding list of words for a given input sentence:

```
def tokenize(sentence, nlp):
  doc = nlp(sentence)
  return [token.vector for token in doc], [token.text for token in doc]
```

We can use this function to recover the contextual embedding vector for a given word in a sentence:

```
def embed_word_contextual(word, sentence, nlp):
  embeddings, words = tokenize(sentence, nlp)
  return embeddings[words.index(word)]
```

This function just demonstrates how to find the right embedding. You will not want to call it repeatedly on the same sentence because it tokenizes every time it is called.

Here is a data structure that indicates a "word" and sets of sentences representing various senses of that word:

```
senses = {"word": "bank",
          "river": [
              "river bank",
              "I buried the money in the river bank",
```

```
            "I buried the money in the bank",
            "on the bank of the Ohio river",
            "on the bank of the Ohio",
            "I slid down the bank",
            "Silt deposits covered the bank",
            "I left my favorite pen on the bank",
            "Fortunately, no one was hurt sliding down the bank",
            "The bank was slippery and treacherous",
          ],
        "money": [
            "money bank",
            "The bank is a safe place to store your wealth",
            "The bank is a safe place to store your money",
            "The savings bank is a safe place to store your money",
            "I deposited the money in the bank",
            "The bank failure caused havoc in the financial markets",
            "I left my favorite pen at the bank",
            "The bank teller lives down by the river",
            "Fortunately, no one was hurt during the bank robbery",
            "The bank robber was slippery and treacherous",
          ],
        }
```

(5 points) Again, use UMAP to create a two-dimensional plot of each of the instances of 'bank' in the above sentences together in a single diagram. Analyze the groupings you find in your plot. Do they make sense? Are the river and money senses of bank adequately separated? Can you identify the commonalities among the various instances of 'bank' in each cluster?

(5 points) Create your own example of a word that has two or more senses. Find ten sentences in `small.txt` containing your word for each of the senses you identify. Repeat your analysis on the examples you generated.

(5 points) For each of the words in the Part A plot, average the contextual embeddings for each occurrence of the word in `small.txt`, then plot the results as you did in Part A. Compare this plot to your plot from Part A. Do the words congregate in the same way? What are the significant differences between the two plots? Can you identify an explanation for any significant differences?

```
bankEmbeddings = []
colors = []
```

```
for sentence in senses['river']:
  doc = nlp(sentence)
  for token in doc:
    if token.text.lower() == "bank":
      bankEmbeddings.append(token.vector)
      colors.append("blue")
```

```
for sentence in senses['money']:
  doc = nlp(sentence)
  for token in doc:
    if token.text.lower() == "bank":
      bankEmbeddings.append(token.vector)
      colors.append("red")
```

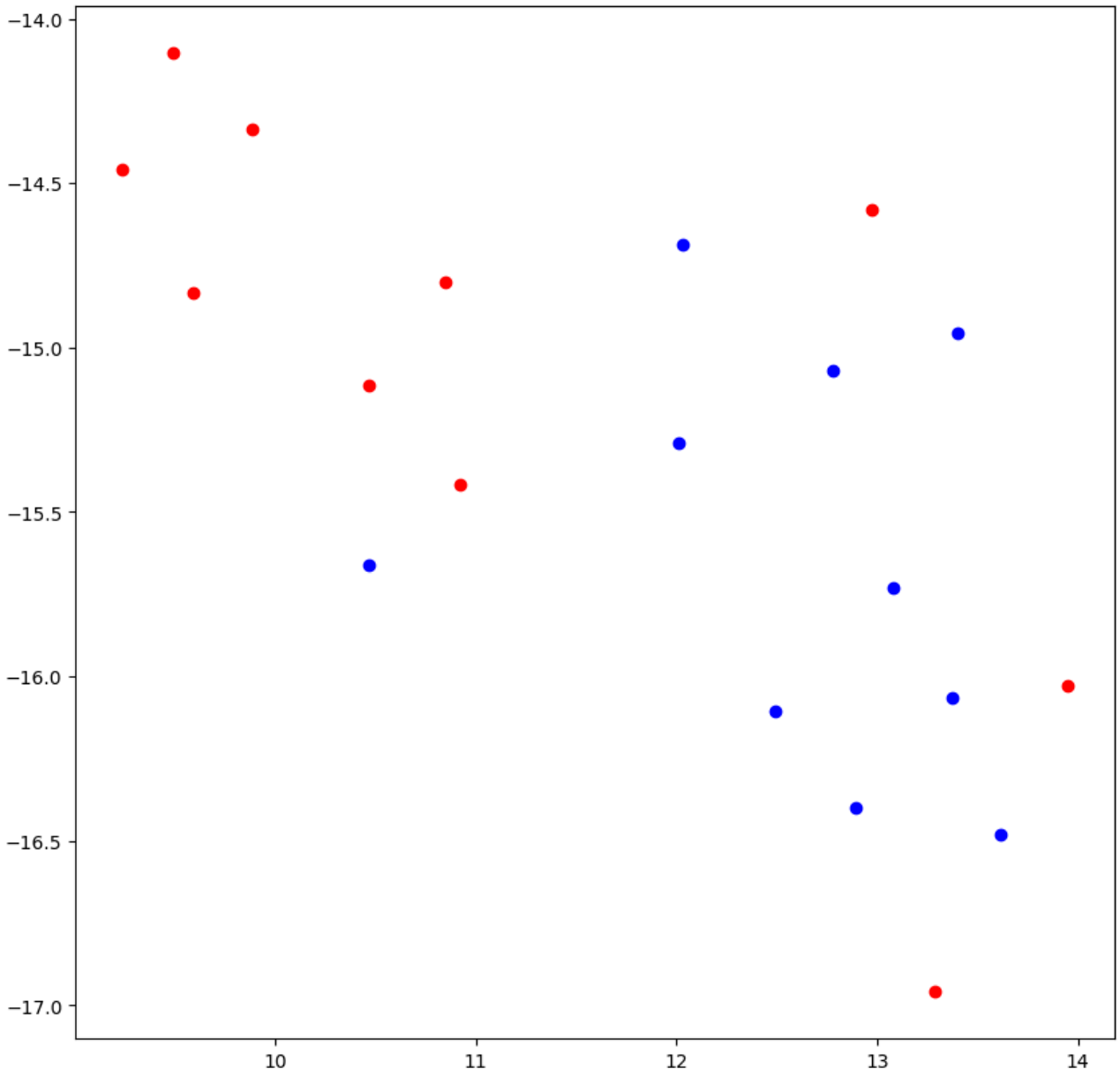
```
bankEmbeddings = np.array(bankEmbeddings)
```

```
reducer = umap.UMAP(n_components=2, random_state=42)
```

```
reducedBankEmbeddings= reducer.fit_transform(bankEmbeddings)
```

```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1
  warn(
```

```
plt.figure(figsize=(10, 10))
for (xy,color) in zip(reducedBankEmbeddings, sense):
  plt.scatter(xy[0],xy[1], c=color)
```



The plot above shows the river sense of bank in blue and the money sense of bank in red. We can see you can group a lot of the red bank embeddings in the top left of the graph and the blue bank embeddings in the bottom right of the bank.

```
lightSenses = {"word": "light",
          "illumination": [
              "Peoples Gas Light & Coke Co., a unit of Chicago-based Peoples Energy Corp., won't have to
              "Light sources are included with our LED, flashing or blinking beacons.",
              "Linear models of interaction of low-level light radiation with turbid media The key eleme
              "Every year the light of Hanukah candles reminds us of the miracle that happened over 2000
              "New motion-sensitive light switches installed at a university are keeping professors on t
              "The light dazzled our eyes; we seemed surrounded by an impenetrable wall of flame.",
              "Only when a mirror is tilted will it reflect the light shining onto it back towards the s
              "These are light sources that vary in intensity at set intervals.",
              "Apparently you can use a bright light, but I never got exactly the results I wanted.",
```

```
            "Light emitting diode street lamps present a very different situation, making up a really
          ],
        "weight": [
            "These are light and elegantly decorated with neutral tones of beige and taupe.",
            "By combining the age-old natural material of timber with specially formulated space-age r
            "The lounge bar offers light snacks, coffees and a selection of wines and beers.With an ir
            "The president has a light schedule this week as he prepares a televised address for next
            "In other economic news on Tuesday, the Dow Jones average of 30 industrial stocks fell 0.9
            "Big Board volume was a light 42.44 million shares at mid-morning.",
            "During the working hours, guests may experience light disturbances.",
            "The stock market gained ground Monday, but trading was so light that most analysts saw li
            "The picture is equally good for light trucks.",
            "Most of Kentucky also got light snowfall overnight.",
          ],
        }
```

```
lightEmbeddings = []
colors = []
```

```
for sentence in lightSenses['illumination']:
  doc = nlp(sentence)
  for token in doc:
    if token.text.lower() == "light":
      lightEmbeddings.append(token.vector)
      colors.append("blue")
```

```
for sentence in lightSenses['weight']:
  doc = nlp(sentence)
  for token in doc:
    if token.text.lower() == "light":
      lightEmbeddings.append(token.vector)
      colors.append("red")
```

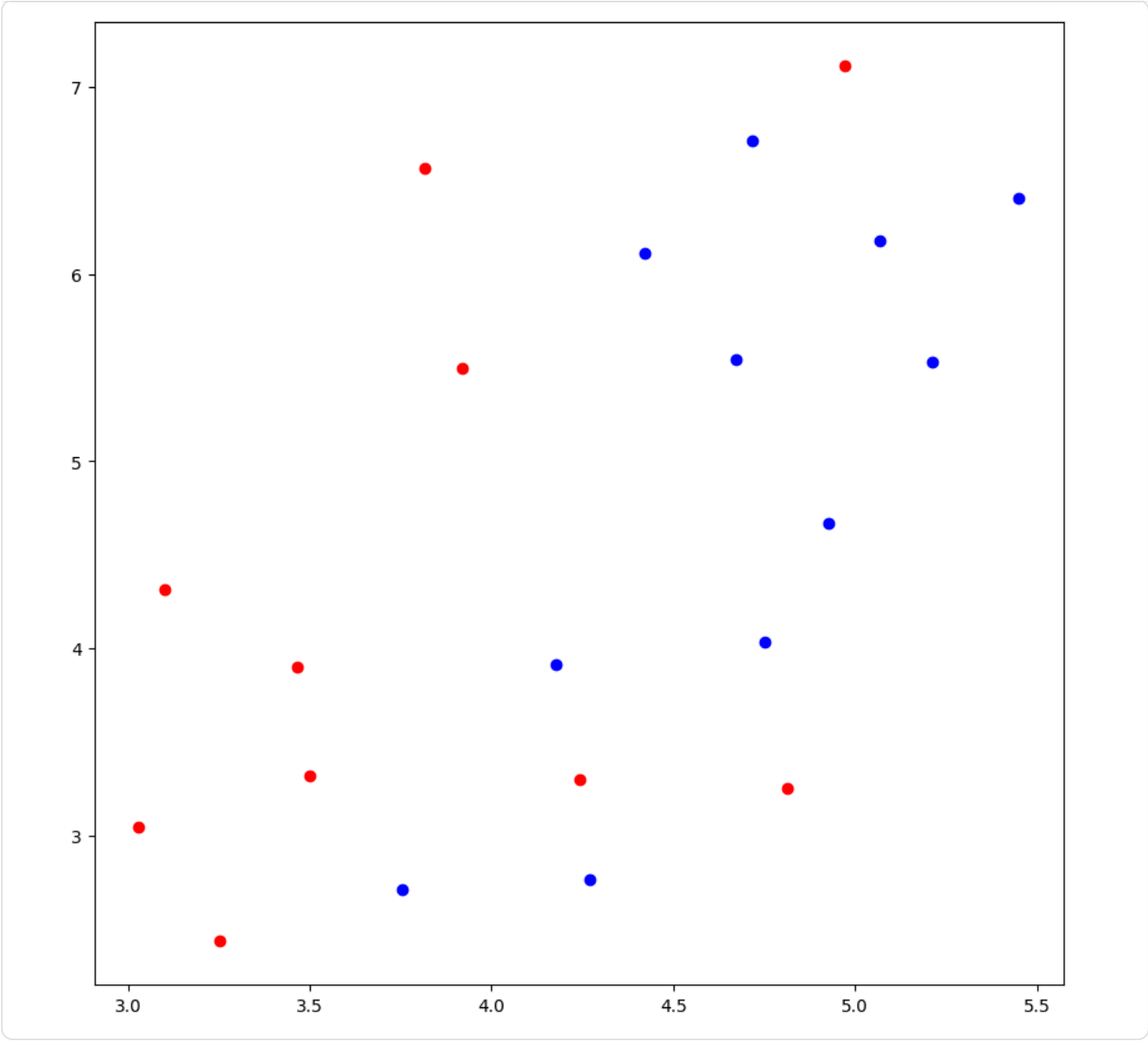
```
lightEmbeddings = np.array(lightEmbeddings)
```

```
lightReducer = umap.UMAP(n_components=2, random_state=42)
```

```
reducedLightEmbeddings= lightReducer.fit_transform(lightEmbeddings)
```

```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1
  warn(
```

```
plt.figure(figsize=(10, 10))
for (xy,color) in zip(reducedLightEmbeddings, colors):
  plt.scatter(xy[0],xy[1], c=color)
```



The blue points represent the illumination meaning of light and the red points represent the weight meaning of light. The blue points tend to be grouped toward the top right of the graph while the bottom left tend to be grouped toward the bottom left of the graph.

```
words_to_visualize = ['paris', 'istanbul', 'moscow', 'france', 'turkey', 'russia',
                      'cat', 'dog', 'chicken', 'duck',
                      'truck', 'train', 'car', 'cars', 'automobile', 'automobiles',
                      'one', 'two', 'three', 'four', '1', '2', '3', '4',
                      'cheese', 'bacon', 'tofu', 'hummus']
```

```
wordContextualEmbeddings = {word:[] for word in words_to_visualize}
```

```
with open('data/small.txt', 'r') as f:
  for line in f:
    doc = nlp(line.lower())
    for token in doc:
      if token.text in words_to_visualize:
        wordContextualEmbeddings[token.text].append(token.vector)
```

```
averagedWordEmbeddings = []
words = []
```

```
for word in words_to_visualize:
  if len(wordContextualEmbeddings[word]) > 0:
    avgEmbedding = np.mean(wordContextualEmbeddings[word],axis=0)
    averagedWordEmbeddings.append(avgEmbedding)
    words.append(word)
```

```
vectorReducer=umap.UMAP(n_components=2, random_state=42)
```

```
reducedEmbeddings= vectorReducer.fit_transform(averagedWordEmbeddings)
```

```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1
  warn(
```

```
plt.figure(figsize=(10, 10))
plt.scatter(reducedEmbeddings[:,0], reducedEmbeddings[:,1])
for i, word in enumerate(words):
    plt.annotate(word,xy=(reducedEmbeddings[i,0], reducedEmbeddings[i,1]))
```


1
2
3